

Notice that this program doesn't have a recursive function; it doesn't have any functions at all! The program emulates recursive function calls by using a list as a stack data structure (stored in the `callStack` variable ❶) to mimic the call stack. A dictionary storing the return address information and `nthNumber` local variable emulates a frame object ❷. The program emulates function calls by pushing these frame objects onto the call stack ❸, and it emulates returning from a function call by popping frame objects off the call stack ❹.

Any recursive function can be written iteratively this way. Although this code is incredibly difficult to understand and you'd never write a real-world factorial algorithm this way, it does demonstrate that recursion has no innate capability that iterative code does not have.